VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Cloud-Based Wildlife Monitoring and Human–Wildlife Conflict Management System

Final Review Report

Submitted By

Rithish K – 24MIC0145

Hemanathan R – 24MIC0066

MTech (Integrated) Computer Science and Engineering

Submitted To

Dr. Jothi K R

Professor Grade 1

Department of Computational Intelligence

School of Computer Science and Engineering

VIT, Vellore

For

CSI3001 – Cloud Computing Methodologies

School of Computer Science and Engineering

October 2026

Abstract

Human–wildlife conflict (HWC) is a major challenge where wildlife habitats overlap with human settlements and agricultural regions. Existing monitoring approaches often depend on a single technology such as camera traps, GPS collars, acoustic sensors or manual observation. These systems generate valuable data, but the information is fragmented and difficult to combine for timely decision-making.

This project presents an integrated wildlife monitoring and HWC management system that combines heterogeneous data sources: GPS, camera observations, motion, acoustic and environmental sensors, historical camera-trap metadata and geographical risk zones. Observations are validated, normalized and correlated by spatial and temporal proximity into wildlife events. Each event is scored by an explainable heuristic risk engine (0–100) using contextual factors such as animal presence, group size, distance from populated areas, movement direction and speed, environmental conditions, sensor agreement and historical conflict.

The system follows a local-first, cloud-ready architecture. The implemented prototype uses a Python-based simulator and controlled replay inputs, MQTT communication support, a FastAPI backend, SQLite with SQLAlchemy, an ETL and data-quality pipeline, GeoJSON risk-zone processing, alert generation and persistence, and an interactive Leaflet/OpenStreetMap dashboard. An optional YOLO11n detector interface is included. Results are demonstrated on controlled replay and simulation data and do not constitute field-validated detection accuracy or calibrated conflict prediction. ESP32 hardware, validated camera AI, cloud deployment, SMS alerts and field validation are identified as future work.

Keywords: Wildlife Monitoring, Human–Wildlife Conflict, IoT, MQTT, Multi-Sensor Integration, Risk Assessment, Cloud Computing, Artificial Intelligence.

1. Introduction

1.1 Background

Wildlife monitoring increasingly relies on camera traps, GPS collars, acoustic and motion sensors and environmental stations. These technologies let researchers and forest authorities collect information about animal presence, movement and conditions over large areas. Camera traps capture images continuously, GPS collars track tagged animals, acoustic and motion sensors give additional evidence of passing animals, and environmental sensors add context that influences movement.

However, these sources produce different data types at different locations and time intervals, so processing each one independently makes it hard to understand the whole situation.

HWC management needs more than detecting an animal. An elephant several kilometres inside a forest is not necessarily an immediate threat. The situation becomes significant when the animal is moving toward a populated area, the distance is shrinking, several sensors confirm its presence and the region has a history of conflict. An integrated system is therefore needed to turn raw observations into meaningful events, evaluate their context and provide appropriate warnings.

1.2 Problem Statement

Current wildlife monitoring systems often suffer from the following problems:

- Wildlife data comes from multiple heterogeneous sources with different formats and characteristics.
- Large volumes of camera and sensor data require significant manual processing.
- AI-based wildlife detection depends on species, location and environmental conditions.
- Remote monitoring locations may have unreliable network connectivity.
- Individual sensor observations rarely give enough context to determine conflict risk.
- Detection, risk assessment and alert generation are often handled as separate functions.
- Observations from different sensors need to be correlated into a common wildlife event.
- Authorities need actionable risk information rather than raw sensor observations.

Problem definition: To develop an integrated wildlife monitoring system capable of collecting heterogeneous observations, correlating them into meaningful wildlife events, assessing contextual HWC risk and generating timely alerts.

1.3 Aim and Objectives

The aim is to build a local-first, cloud-ready prototype that integrates multi-source wildlife observations, event correlation, contextual risk assessment, alerting and a monitoring dashboard. The objectives are:

1. To ingest, validate and normalize observations from multiple source types into a common representation.
2. To correlate observations into wildlife events using spatial and temporal proximity.
3. To compute an explainable contextual risk score with geographical risk-zone context.
4. To generate, persist and display risk-based alerts.
5. To provide a dashboard with map-based event, risk and alert visualization.
6. To evaluate the software pipeline using automated tests and controlled replay scenarios.
7. To identify the remaining steps (hardware, validated AI, cloud, field validation) for real-world deployment.

1.4 Scope

The project demonstrates a software workflow for wildlife observation processing and risk prioritization. It does not claim that real-world incidents will be prevented, nor that the system has been validated for field deployment. Reliability of detection, risk assessment and warnings depends on input data quality, model performance, sensor accuracy and network availability.

1.5 Contributions

- A common observation model with ETL, duplicate detection, invalid-record handling and provenance tracking.
- Backend event correlation using time and location instead of sensor-generated event IDs.
- An interpretable heuristic risk engine with GeoJSON polygon and multipolygon risk-zone support.
- Alert generation, persistence and history, with an interactive map dashboard.
- A local-first design that validates the full logical pipeline before cloud migration and hardware integration.

2. Literature Review

2.1 Literature Survey

AI-based camera-trap processing. Tabak et al. investigated machine-learning classification of animal species in camera-trap images, showing that AI can cut the manual effort of processing large image sets. Vélez et al. evaluated platforms for processing camera-trap data with AI and noted differences in workflows, model performance and processing requirements. Zampetti et al. studied automated wildlife density estimation using camera traps, showing how automated classification supports population estimation.

Automated wildlife monitoring. Mitterwallner et al. examined automated visitor and wildlife monitoring with camera traps and machine learning. The work shows the usefulness of automated detection but also variation caused by location, season and environmental conditions.

Real-time wildlife alerts. Dertien et al. (TrailGuard AI) demonstrate edge-based camera detection with real-time alerts for conflict mitigation.

Multi-modal monitoring. Brickson et al. review AI for elephant monitoring across camera, acoustic, GPS, seismic, aerial and satellite sensing. Each modality has distinct information and limits, and combining them gives a more complete picture.

IoT and edge systems. Raiaan et al. present an ESP32-CAM and YOLOv8 based detection system for agricultural farm security, and other reviewed systems use MQTT, embedded hardware and edge processing in remote settings with limited connectivity.

Spatial risk modelling. GIS and MaxEnt studies identify areas of higher human–elephant conflict risk from environmental and occurrence data, but focus on spatial prediction rather than continuous multi-sensor correlation and alerting. Muliya et al. describe a satellite-telemetry and mass-communication early warning system for human–elephant coexistence in Southern India.

2.2 Summary Table

S.No.	Paper / Study	Methodology	Key Contribution	Limitation / Gap
1	Tabak et al. (2019)	Machine / deep learning	Automated species classification from camera-trap images	Performance can drop outside training locations
2	Vélez et al. (2023)	AI camera-trap platforms	Compared AI platforms for image processing	Performance and workflow requirements vary
3	Zampetti et al. (2024)	Camera traps + AI + density estimation	Automates classification and density estimation	Classification errors affect population estimates
4	Mitterwallner et al. (2023)	Camera traps + MegaDetector / ML	Automated monitoring of wildlife and visitors	Detection varies with site, season, conditions
5	Dertien et al. (2023)	Edge AI + camera + real-time alerts	Real-time detection and conflict mitigation	Focused on specific camera-based scenarios
6	Brickson et al. (2023)	AI + camera, acoustic, seismic, aerial, satellite	Reviews AI across elephant monitoring modalities	Data transferability, annotation, multimodal integration
7	Thornton et al. (2026)	AI classification + occupancy modelling	Compared AI and expert classification	Classification uncertainty and species/site differences
8	Raiaan et al. (2023)	ESP32-CAM + YOLOv8 + cloud alerts	Real-time animal detection on farms	Animal intrusion focus, not broader HWC risk
9	Edge-AI wildlife monitoring studies	Edge AI + MQTT/TCP-IP + sensors	Combines sensing, AI, communication	Hardware, deployment, multi-sensor integration are hard
10	GIS / MaxEnt HWC studies	GIS + MaxEnt + environmental variables	Predicts spatial distribution of HWC risk	Spatial prediction only; no real-time sensor-to-alert pipeline
11	Real-time HWC alert systems	Detection + communication	Rapid warning after detection	Detection alone does not represent contextual risk
12	Multi-sensor wildlife research	GPS + camera + acoustic + environmental	Richer information from combined modalities	Needs synchronization and unified event representation

3. Existing System Analysis

Existing practical systems demonstrate the usefulness of individual technologies, but each has limits.

S.No.	Existing System	Technology	What It Provides	Limitation
1	Camera-trap AI monitoring	Camera traps + AI / deep learning	Automatic detection and species classification	Depends on coverage, image quality, model and site
2	GPS-based tracking	GPS collars + telemetry	Movement and location of tagged animals	Cannot monitor uncollared animals
3	TrailGuard AI	Edge AI + camera + real-time comms	Detection and rapid alerts	Camera-centred, specific deployments
4	IoT monitoring systems	IoT sensors + MQTT + backend	Continuous remote sensing	Connectivity and device reliability in remote areas
5	ESP32-CAM + YOLO systems	ESP32-CAM + YOLOv8	Real-time animal detection and alerts	Detection-focused, no full HWC risk analysis
6	Acoustic monitoring	Microphones + acoustic AI	Detects vocalizations and activity	Affected by noise and distance
7	GIS / MaxEnt HWC systems	GIS + MaxEnt	Predicts conflict-prone areas	Spatial modelling, not continuous monitoring
8	Multi-modal elephant monitoring	Camera + GPS + acoustic + seismic + satellite	Complementary information	Requires synchronization and integration
9	Edge-AI monitoring	Edge devices + AI inference	Near-real-time local processing	Limited by hardware, power, model complexity
10	Automated HWC alert systems	Detection + notification	Rapid warnings	Does not quantify contextual conflict risk

4. Research Gap

1. **Fragmented data:** cameras, GPS devices, acoustic and motion sensors and environmental stations are processed separately.
2. **Limited multi-sensor integration:** observations are not always merged into a single event representation.
3. **Event correlation challenge:** a stationary camera or motion sensor does not know which animal or event caused its observation.
4. **Limited contextual risk assessment:** detection alone does not indicate the level of conflict risk.
5. **Connectivity challenges:** remote regions cannot assume continuous Internet connectivity.
6. **Detection-to-alert gap:** approaches rarely cover the complete path from observation to risk assessment and actionable alert.

7. **Scalability:** a system should support multiple animals, sensor types, locations and events without isolated components.

The primary contribution of this project is the integration of these functions into one pipeline, not a new detection algorithm.

5. System Design

5.1 Proposed Architecture

The system follows a local-first, cloud-ready architecture in seven layers: (1) data sources (IoT sensors, camera/AI observations, GPS/telemetry, historical records, environmental data); (2) a communication layer (MQTT for real-time messages, HTTP/REST for other sources); (3) data ingestion (receiver, validation, standardization); (4) data management (a central database of wildlife, environmental and conflict data); (5) an analytics and risk engine (spatial and temporal analysis, risk assessment); (6) a dashboard and alerts layer; and (7) users such as forest officials, researchers, local communities and policy makers. Cross-cutting concerns are security, data quality, scalability, system monitoring and backup. The overall flow is Collect → Integrate → Analyze → Assess risk → Alert → Take action.

5.1.1 Target Cloud Deployment Architecture

1. **Field monitoring layer:** a wildlife camera (camera trap), HC-SR04 ultrasonic sensor, DHT11/DHT22 temperature and humidity sensor, and an ESP32 microcontroller that reads sensor data and sends it to the cloud as JSON over HTTPS/MQTT.
2. **Edge processing layer:** a Raspberry Pi acts as the local processing unit and runs a YOLOv8 model that detects and classifies wildlife in real time. Each detection result carries species, confidence, image and timestamp.
3. **Cloud computing layer:** a cloud server (AWS EC2 or cloud VM) hosts a FastAPI REST API that receives and validates sensor and detection data, a risk assessment module that generates a Low, Medium or High risk level and creates alert events, a MySQL/PostgreSQL database for detections, sensor data, locations, risk events and system logs, and an optional notification service for Email/SMS alerts.
4. **Web application layer:** a React.js/HTML/CSS/JS dashboard showing the live camera feed and detections, sensor data, risk levels and alerts, a map of detection locations, historical records and device settings.
5. **Users and response layer:** forest department officials, farmers and local communities receive alerts when risk is high.

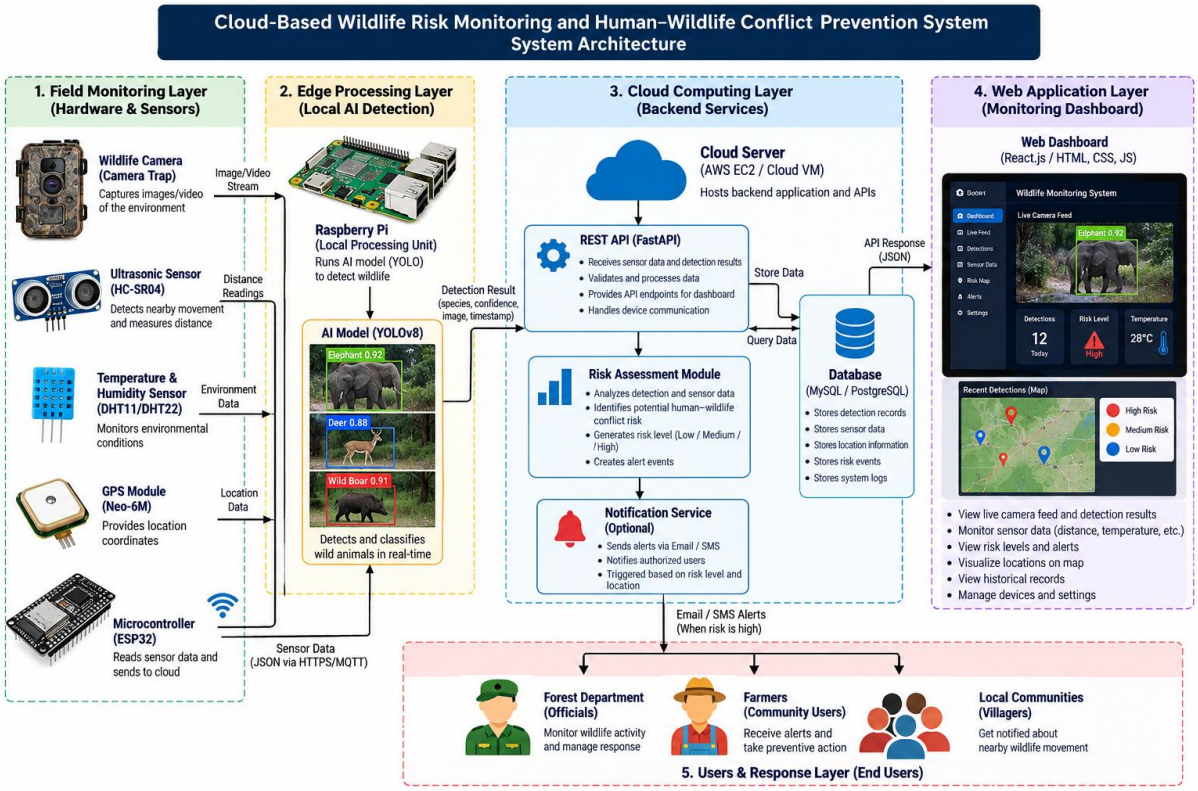


Figure 2: Target cloud-based system architecture (field monitoring, edge processing, cloud computing, web application and user response layers).

The current prototype (Section 6) implements the backend, risk, alert and dashboard logic locally using simulated and replayed data. The ESP32 and sensor hardware, Raspberry Pi edge inference, cloud hosting and Email/SMS notifications shown in Figure 2 are the planned deployment stages (Section 9).

5.2 Major Modules

- **Data collection:** GPS collars, camera/AI observations, motion, acoustic, environmental and historical data.
- **Communication:** MQTT/HTTP transport of observations.
- **Ingestion and standardization:** validation, normalization and a common event-oriented representation.
- **Event correlation:** associates observations by temporal and spatial proximity rather than sensor-generated event IDs.
- **Data management:** stores raw observations, integrated events, risk results and alerts.
- **Analytics and risk:** derives movement, distance, sensor agreement, environmental and historical context.
- **Dashboard and alerts:** visualizes risk and produces actionable warnings.

5.3 UML Diagrams

[Insert Figure 3: UML use-case view showing interaction between data sources, the monitoring platform and stakeholders (ingest sensor data, store observations, correlate events, analyse context, calculate risk, view dashboard, generate alert).]

[Insert Figure 4: UML activity flow: Receive observation → Validate and standardize → Correlate by time and location → Integrate multi-sensor context → Calculate contextual risk → Generate dashboard result or alert.]

5.4 Key Design Decision: Backend Event Correlation

The design deliberately avoids embedding an event identifier in every raw sensor payload. In a real deployment a camera or motion sensor knows only its fixed location and what it detected, and a GPS collar knows only its tagged animal. The backend therefore resolves which observations belong to the same real-world event using time and spatial proximity with configurable matching parameters. This supports concurrent animals and heterogeneous sensors while keeping raw sensor messages independent.

6. Implementation

6.1 Implementation Status

Module	Technology / Implementation	Status
Data ingestion	Source adapters, controlled replay, simulated observations	Implemented
ETL pipeline	Validation, normalization, common observation representation	Implemented
Data quality	Duplicate detection, invalid-record handling, provenance tracking	Implemented
Communication	MQTT publish/subscribe support	Implemented
Backend	FastAPI + MQTT subscriber	Implemented
Storage	SQLite + SQLAlchemy	Implemented
Event correlation	Configurable spatial and temporal matching	Implemented
Risk-zone processing	GeoJSON polygon and multipolygon support	Implemented
Risk engine	Explainable heuristic score (0–100)	Implemented
Alerts	Risk-based generation, persistence, history	Implemented
Dashboard	Leaflet / OpenStreetMap interactive map and event visualization	Implemented
Historical camera-trap data	Manifest of 1,000 records	Future implementation
ESP32 / real sensors	Physical hardware integration	Implemented
SMS / mobile delivery	External notification service	Future stage
Cloud deployment	Cloud-hosted backend and storage	Implemented
Field validation	Evaluation with verified real-world observations	Future stage

6.2 Observation Model and Data Pipeline

A common observation model processes data from supported source adapters, controlled replay inputs, simulated observations and MQTT inputs. Each observation is validated and normalized into a consistent structure. Duplicates are identified, invalid records are handled, and the source of each record is kept for traceability. In Review-1 the simulator represented several wildlife groups at once, each with species, group count, location, speed, heading and behaviour, with virtual camera, GPS, motion, acoustic and environmental devices deriving correlated observations from the shared state. Device health (battery, signal strength, connection status, reading drops) was also simulated. The example zone contains a village, forest edge and agricultural area, with one elephant herd starting about 5 km from the village and biased toward it.

6.3 Event Correlation

The correlation module groups observations that are close enough in time and location into a common wildlife event. Observations that do not meet the configured criteria stay separate, so different animals, repeated tracks of the same animal, nearby sensor matches and expired tracks are handled distinctly.

6.4 Risk Engine

The risk engine is an interpretable heuristic module, not a trained predictive model. It produces a 0–100 risk-prioritization score from the available evidence for a correlated event: animal presence and count, distance to the village, movement direction and speed, environmental conditions, acoustic and motion confirmation, historical conflict, observation recency and proximity to configured GeoJSON risk zones. Sensor agreement and data quality reduce confidence when observations conflict or devices are degraded. The score maps to LOW, MEDIUM, HIGH or CRITICAL. It is not a calibrated probability of conflict, and its usefulness for real incidents requires validation against verified historical data.

6.5 Alert Generation

After risk assessment, the alert module generates alerts for configured risk levels and stores them in the local database. The backend serves active alerts and alert history to the dashboard. Alerts are currently software-level only; SMS and mobile delivery to forest officials and communities is future work.

6.6 Backend, Storage and Dashboard

The FastAPI backend receives observations (including through an MQTT subscriber), runs correlation and risk assessment, persists observations, events, risk results and alerts through SQLite and SQLAlchemy, and exposes processed information over its API. The dashboard

retrieves this data and provides a map view with GeoJSON risk zones, event history, risk-factor inspection, alert monitoring and source-level inspection of observation details.

7. Testing and Results

7.1 Testing Approach

The pipeline was evaluated with automated tests and controlled replay scenarios covering: observation validation and normalization; duplicate detection and invalid-record handling; spatial–temporal correlation and separation of non-matching observations; risk-score generation and availability of contextual factors; GeoJSON risk-zone processing; database persistence and retrieval of observations, events, risk results and alerts; alert generation and history; backend API and dashboard integration; and handling of replay and simulation inputs. Earlier simulation scenarios also checked that movement toward the village raised risk, sensor disagreement lowered the agreement score, degraded devices lowered data quality, and no-wildlife conditions produced LOW risk with no alert.

7.2 Results Summary

Component	Result
Data ingestion	Source adapters, controlled replay and simulation inputs provide observations
ETL processing	Observations validated and normalized into a common representation
Data quality	Duplicate detection, invalid-record handling and provenance tracking supported
Event correlation	Related observations associated using spatial and temporal relationships
Risk-zone processing	GeoJSON polygon and multipolygon zones provide geographical context
Risk assessment	Interpretable heuristic score generated from contextual information
Local storage	Observations, events, risk results and alerts persisted in SQLite
Backend	FastAPI exposes processed information to the dashboard
Alert management	Alerts generated, stored and retrievable
Dashboard	Map, event history, risk factors and alerts visualized
MQTT communication	Subscriber and support implemented; live broker connectivity to be verified

7.3 Discussion

The results show that the principal software modules work together on controlled replay and simulation data. They do not establish field-level detection accuracy, real-time hardware performance or calibrated conflict-prediction accuracy. The displayed outputs are software-generated and must not be read as verified real-time detections.

8. Security, Reliability and Limitations

Security considerations: protect credentials and avoid hardcoded secrets, authenticate API access, validate incoming requests, restrict exposed ports and database services, and use secure transport in production. These are design recommendations for the cloud stage; the current prototype runs locally.

Reliability: depends on device availability, sensor accuracy, network connectivity and backend error handling. Buffering, retry and synchronization are planned for intermittent connectivity.

Limitations:

1. Results come from controlled replay and simulated data, not live field observations.
2. Physical sensors (ESP32, GPS, motion, acoustic) are not yet integrated.
3. Camera-AI inference (YOLO11n) is not yet validated on real images.
4. The risk score is a heuristic and not calibrated against verified incidents.
5. Live MQTT broker communication and historical dataset sources need verification.
6. Alerts are dashboard-level only; no external notifications.
7. The system is not yet deployed on cloud infrastructure and has not had scalability or security testing.

9. Future Work

Future Area	Planned Work
Real IoT integration	Integrate ESP32 devices and physical GPS, motion, acoustic and environmental sensors
Real camera-trap AI	Validate detection, species classification and counting on real camera-trap images
Cloud deployment	Deploy backend and database on cloud infrastructure for remote access and scalable processing
Advanced GIS and spatial analysis	Extend map visualization and risk zones using verified locations, conflict records and movement data
Advanced risk prediction	Evaluate and calibrate the heuristic score using verified historical observations and HWC datasets
Connectivity and edge support	Buffering, retry and synchronization for intermittent networks
SMS / mobile alerts	Deliver authorized warnings to forest officials and relevant communities
Historical data validation	Verify source, image availability, timestamps and geography of camera-trap datasets
Real-world validation	Evaluate the pipeline with physical sensor observations and documented HWC incidents

10. Conclusion

The project addresses the gaps found in the literature and existing-system analysis by combining observation processing, data validation, spatial–temporal event correlation, contextual risk assessment, persistent storage and alert management in one workflow. Over the two review stages, the local prototype grew from a simulated MQTT, FastAPI and SQLite pipeline into a system with an ETL pipeline, source adapters, duplicate detection, provenance tracking, GeoJSON risk-zone processing and an interactive geographical dashboard.

The workflow is demonstrated with controlled replay and simulation data, with supporting historical camera-trap metadata. The risk score remains a heuristic assessment, not a calibrated prediction of conflict. Physical sensor integration, verified live ingestion, validated AI detection, cloud deployment and field evaluation remain future activities. The modular, local-first approach gives a foundation for further development and testing before real-world deployment.

References

[1] Tabak, M. A., Norouzzadeh, M. S., Wolfson, D. W., Sweeney, S. J., Vercauteren, K. C., Snow, N. P., et al. (2019). Machine learning to classify animal species in camera trap images: Applications in ecology. *Methods in Ecology and Evolution*, 10(4), 585–590. https://doi.org/10.1111/2041-210X.13120

[2] Norouzzadeh, M. S., Nguyen, A., Kosmala, M., Swanson, A., Palmer, M. S., Packer, C., & Clune, J. (2018). Automatically identifying, counting, and describing wild animals in camera-trap images with deep learning. *PNAS*, 115(25), E5716–E5725. https://doi.org/10.1073/pnas.1719367115

[3] Gomez Villa, A., Salazar, A., & Vargas, F. (2017). Towards automatic wild animal monitoring: Identification of animal species in camera-trap images using very deep convolutional neural networks. *Ecological Informatics*, 41, 24–32. https://doi.org/10.1016/j.ecoinf.2017.07.004

[4] Vélez, J., McShea, W., Shamon, H., Castiblanco-Camacho, P. J., Tabak, M. A., Chalmers, C., Fergus, P., & Fieberg, J. (2023). An evaluation of platforms for processing camera-trap data using artificial intelligence. *Methods in Ecology and Evolution*, 14, 459–477.

[5] Zampetti, A., Mirante, D., Palencia, P., & Santini, L. (2024). Towards an automated protocol for wildlife density estimation using camera-traps. *Methods in Ecology and Evolution*.

[6] Mitterwallner, V., et al. (2023). Automated visitor and wildlife monitoring with camera traps and machine learning. *Remote Sensing in Ecology and Conservation*. https://doi.org/10.1002/rse2.367

[7] Whytock, R. C., Świeżewski, J., Zwerts, J. A., Bara-Słupski, T., Koumba Pambo, A. F., Rogala, M., et al. (2021). Robust ecological analysis of camera trap data labelled by a machine learning model. *Methods in Ecology and Evolution*, 12(6), 1080–1092. https://doi.org/10.1111/2041-210X.13576

[8] Dertien, J. S., Negi, H., Dinerstein, E., Krishnamurthy, R., Negi, H. S., Gopal, R., et al. (2023). Mitigating human–wildlife conflict and monitoring endangered tigers using a real-time camera-based alert system. *BioScience*, 73, 748–757. https://doi.org/10.1093/biosci/biad076

[9] Brickson, L., et al. (2023). Elephants and algorithms: A review of the current and future role of AI in elephant monitoring. *Journal of the Royal Society Interface*, 20, 20230367. https://doi.org/10.1098/rsif.2023.0367

[10] Raiaan, M. A. K., Fahad, N. M., Chowdhury, S., Sutradhar, D., Mihad, S. S., & Islam, M. M. (2023). IoT-Based Object-Detection System to Safeguard Endangered Animals and Bolster Agricultural Farm Security. *Future Internet*, 15(12), 372. https://doi.org/10.3390/fi15120372

[11] Muliya, S. K., et al. (2026). Satellite-telemetry and mass communication based early warning system for human-elephant coexistence in Southern India's agroforestry landscapes. *Frontiers in Conservation Science*, 6, 1640741. https://doi.org/10.3389/fcosc.2025.1640741

[12] Thornton, D., et al. (2026). Identification of camera trap images by artificial intelligence and human experts produces similar multi-species occupancy models. *Journal of Applied Ecology*, 63, e70370. https://doi.org/10.1111/1365-2664.70370

[13] Delisle, Z. J., Flaherty, E. A., Nobbe, M. R., Wzientek, C. M., & Swihart, R. K. (2021). Next-generation camera-trapping: Systematic review of historic trends suggests keys to expanded research applications in ecology and conservation. *Frontiers in Ecology and Evolution*, 9.

[14] Green, S. E., Rees, J. P., Stephens, P. A., Hill, R. A., & Giordano, A. J. (2020). Innovations in camera-trapping technology and approaches: The integration of citizen science and artificial intelligence. *Animals*, 10(1), 132. https://doi.org/10.3390/ani10010132

[15] Fennell, M., Beirne, C., & Burton, A. C. (2022). Use of object detection in camera trap image identification. *Global Ecology and Conservation*, 35, e02104. https://doi.org/10.1016/j.gecco.2022.e02104

[16] Kays, R., Cove, M. V., Diaz, J., Todd, K., Bresnan, C., Snider, M., et al. (2022). SNAPSHOT USA 2020: A second coordinated national camera trap survey of the United States during the COVID-19 pandemic. *Ecology*, 103(10), e3775. https://doi.org/10.1002/ecy.3775

[17] Schneider, S., Greenberg, S., Taylor, G. W., & Kremer, S. C. (2020). Three critical factors affecting automated image species recognition performance for camera traps. *Ecology and Evolution*, 10(7), 3503–3517. https://doi.org/10.1002/ece3.6147